

Exercise Sheet 9

Efficient Computing/Parallel Computing 2 in SS 2026

Date of issue: 23.6.2026

Deadline: 29.6.2026

The exercises marked with ★ are relevant for admission to the examination and must be submitted in electronic form in Moodle by the submission deadline.

Exercise 9.1 (*CUDA vector scaling*).

Write a CUDA program that computes the scaled version $\mathbf{y} \leftarrow \alpha \mathbf{x}$ for a given scalar $\alpha \in \mathbb{R}$ and vector $\mathbf{x} \in \mathbb{R}^n$ of dimension $n = 1024 \cdot i$, where i is passed as a command line argument.

- Write a CUDA kernel `vector_scale`. Each thread shall compute one vector element `y[i] = alpha * x[i]`.
- Choose a one-dimensional block and grid configuration. Make sure that the kernel works even if the number of threads in the grid is larger than n .
- Write the necessary code: allocate memory on the host and on the device, initialize $\mathbf{x}$, copy data to the device, start the kernel, and copy $\mathbf{y}$ back to the host.
- Verify the computed result by printing a few values and make runtime measurement for suitable dimensions and discuss the results.

Exercise 9.2 (*CUDA dot product with shared memory*).

Write a CUDA program that computes the dot product

$$s \leftarrow \mathbf{x}^T \mathbf{y} = \sum_{i=0}^{n-1} x_i y_i$$

for two given vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ of dimension $n = 1024 \cdot i$, where i is passed as a command line argument.

- Write a CUDA kernel `dot_product`. Each thread shall compute one product `x[i] * y[i]`.
- Use shared memory inside each thread block to reduce the products of the block to one partial sum.
- Add the partial sum of each block to one global result using `atomicAdd`.
- Write the necessary code: allocate memory for both vectors and the result on the host and on the device, initialize both vectors, copy data to the device, set the device result to zero, start the kernel, and copy the result back to the host.
- Make runtime measurement for suitable dimensions and discuss the results.